

Name: Shrisarth Sunil kutwad Class: SOC 8 Batch: ~~22~~ B

Subject: IDS Roll No.: 28 Exp. No.: 2

Name of the Experiment: _____

Performed on: _____

Submitted on: _____

Marks	Teacher's Signature with date
10	

IDS Assignment - 2

1. Sequential organization & characteristics of linear data structures

It is a method of storing data elements in a continuous memory location where elements are arranged one after another in a fixed order.

Characteristics:

- elements are stored in contiguous memory locations.
- each element has a unique index or position
- Direct access to elements is possible using indexing
- Data is arranged in a linear sequence.

Incomplete for Theory ☐ Diagrams ☐ Observation Table ☐

Calculations ☐ Graphs ☐ Results ☐ Conclusion ☐

Understanding Through Q/A ☐ Late Submission ☐ Neatness ☐

Teacher's Signature

2.2 Explain how indexing and Slicing work in python sequences. Illustrate how they differ when applied to Strings and lists.

- Indexing in python sequences.
indexing is used to access a single element from a sequence.
- Indexing starts from 0 (left to right)
 - Negative indexing starts from -1 (right to left)

Ex:- Code

```
S = "Python"
```

```
l = [10, 20, 30, 40]
```

```
Print(S[0])
```

```
Print(S[-1])
```

```
Print(l[1])
```

```
Print(l[-2])
```

- Slicing in python sequences.
Slicing is used to extract a part (subsequence of a sequence).

- Syntax:

Sequence [start: stop: step]

start → starting index (included)

stop → Ending index (excluded)

step → gap between elements (optional)

Ex. Print(S[1:4])
Print(L[0:3])
Print(S[::2])
Print(L[:: -1])

- Q.3 write a python program that:
- uses indexing and slicing on string list, and.
 - Applies at least two built-in functions to display meaningful result.

→ Python Code:-

```
text = "Python programming"  
# index
```

```
first_char = text[0]
```

```
last_char = text[-1]
```

```
# Slicing
```

```
sub_text = text[0:6]
```

```
every_second = text[::2]
```

```
length = len(text)
```

```
max_char = max(text)
```

```
Print("Original string:" + text)
```

```
Print("First Character:" + first_char)
```

```
Print("Last Character:" + last_char)
```

```
Print("Sliced string(0:6):" + sub_text)
```


2. Array as an Abstract Data type (ADT)

ADT is collection of elements of the same data type stored in contiguous memory locations & accessed using an index.

• Components of ADT:

- Elements: Data items of the same type.
- index: Position used to access elements.
- Size: Total number of elements.

• Basic operations on array:

- Transversal: visiting each element.
- Insertion: Adding a new element.
- Deletion: Removing an element.
- Searching: finding an element.

3. Insertion, deletion & Searching in a 1D array.

Consider an array:

$A = [10, 20, 30, 40]$

insertion: insert ~~in~~ is on position 2?

Shift elements right $\rightarrow A = [10, 20, 25, 30]$

- Deletion: Delete element at position

Shift elements left $\rightarrow A = [10, 20, 25, 40]$

- Searching: To Search 25

Compare sequentially until found

25 is found in index 2

4. Advantages & limitations of arrays

Advantages:

- Fast direct access using index (O(1))
- simple easy to implement
- efficiently memory usage for fixed size data
- good for storing homogeneous data

Limitations:

- Fixed Size
- insertion & deletion are time-consuming
- Wastage of memory if size not used